

Artifact Two:

Algorithms and Data Structures Narrative

The artifact that I selected for the second enhancement algorithms and data structure was my project from CS 210 which is my item tracker project. I created the original project on February 23, 2025. This project reads items from an input file and tracks the frequency of the items. This program allows users to search for the frequency of an item, print a list of all items and their frequency, print a histogram that uses asterisks, and save the data to an output file. Originally the project was written in C++ using OOP.

I chose this item because I believe it allows me to demonstrate my knowledge of algorithms and data structures. For the first artifact, I implemented the hash table but focused on the conversion from C++ to Python. In the original artifact, the program used a C++ map which stored the grocery items and the frequency of those items. This enhancement allows me to improve the efficiency of item tracking and lookup of items. I used this program for the first artifact which I converted from its original language C++ to Python using a hash table-based dictionary. This enhancement improved the average lookup and insertion time which was $O(\log n)$ in the original to $O(1)$ in the enhancement. In this artifact, I also added a function (`def normalize_item()`) for input normalization, making it easier to reuse and improve maintainability. The normalization allows the user to type in an item regardless of capitalization and produce accurate results.

I believe I met the course outcome that I said aligned with my enhancement, which was outcome 3. This outcome focused on algorithmic principles and computer science practices. With this enhancement, I needed to go through the original data structure and decide if another

data structure would improve the original. I decided that a hash table Python dictionary would improve the lookup efficiency compared to the C++ tree-based map from the original project. I was able to achieve this with the code `self.item_frequency[item] = self.item_frequency.get(item, 0) + 1` which allows the program to read through the file, retrieve the frequency of the item, and updates the count which updates at an average lookup of $O(1)$.

Since I am using the same artifact for two of the categories, I was able to implement this during the first one. That being said, what I learned when implementing the algorithm is that it gave me a better understanding of how selecting a data structure can impact a program. Though the original in C++ and the enhancement in Python produce the same results, they are different in the lookup and insertion efficiency. It is important to understand different algorithms and time complexity, which helped me to understand better. The challenges that I faced with this artifact were showing that I can enhance the original project while maintaining its original functionality. Another challenge was normalizing the user's input, so it would treat the input value the same whether it was uppercase, all lowercase, or a mixture of both. This was important because if it treats the same word as different entries based on capitalization then it would produce inaccurate results. In order to resolve this issue, I created a function (`normalize_item()`) and also used `strip().lower()` to make sure that the users input and the stored value were formatted the same before the structure compared the input to the data. Continuing to work on the same artifact helps me to solidify my understanding of programming languages, data structures, and algorithm efficiency.